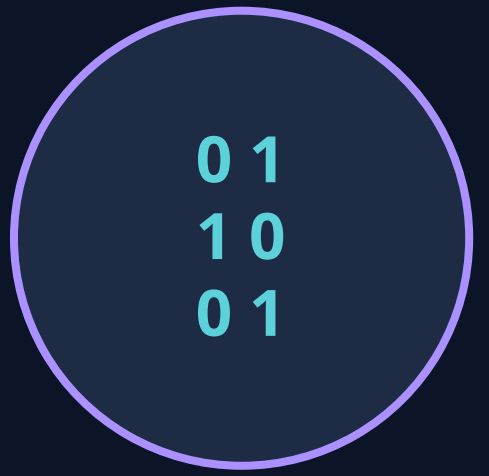
DATA STRUCTURES & ALGORITHMS

LOSSLESS DATA COMPRESSION



Fixed-length → RLE → Huffman → Canonical Huffman → LZ77

THEORY + NOTEBOOK + EMPIRICAL EVIDENCE

Group: [names to add] | Submission deadline: [verify]

What must be submitted

The handwritten component is indispensable; code and slides must agree with it.

HANDWRITTEN SOLUTIONS

Q1–Q5: every calculation, heap merge, bitstream trace, canonical assignment, sliding-window trace, and complexity analysis.

WORKING CODE

All five compression/decompression pairs in Python, 20 required dataset round trips, plots and reproducible notebook.

SLIDE PRESENTATION

Readable academic explanation of methods and results, plus an actual link to the team GitHub repository.

CRITICAL: Without handwritten work, the professor will not consider the submission.

Five stages of increasing sophistication

Each version targets a limitation left by its predecessor.

V1

FIXED

Same bits for all symbols

V2

RLE

Consecutive identical runs

V3

HUFFMAN

Statistical frequency skew

V4

CANONICAL

Codes reconstructed from lengths

V5

LZ77 + HUFF

Repeated phrases within a window

Lossless means: decompress(compress(data)) == data — for every valid input.

Theoretical comparison: 100 symbols

Alphabet: A45, B13, C12, D16, E9, F5 — six distinct symbols.

V1 • fixed-length

300 bits

V2 • 6 contiguous runs

60 bits

V2 • 100 separate runs

1,000 bits

V3 / V4 • Huffman

224 bits

Shannon entropy

$$H(X) = -\Sigma\, p_i \log_2(p_i) = 2.219880 \text{ bits/symbol}$$

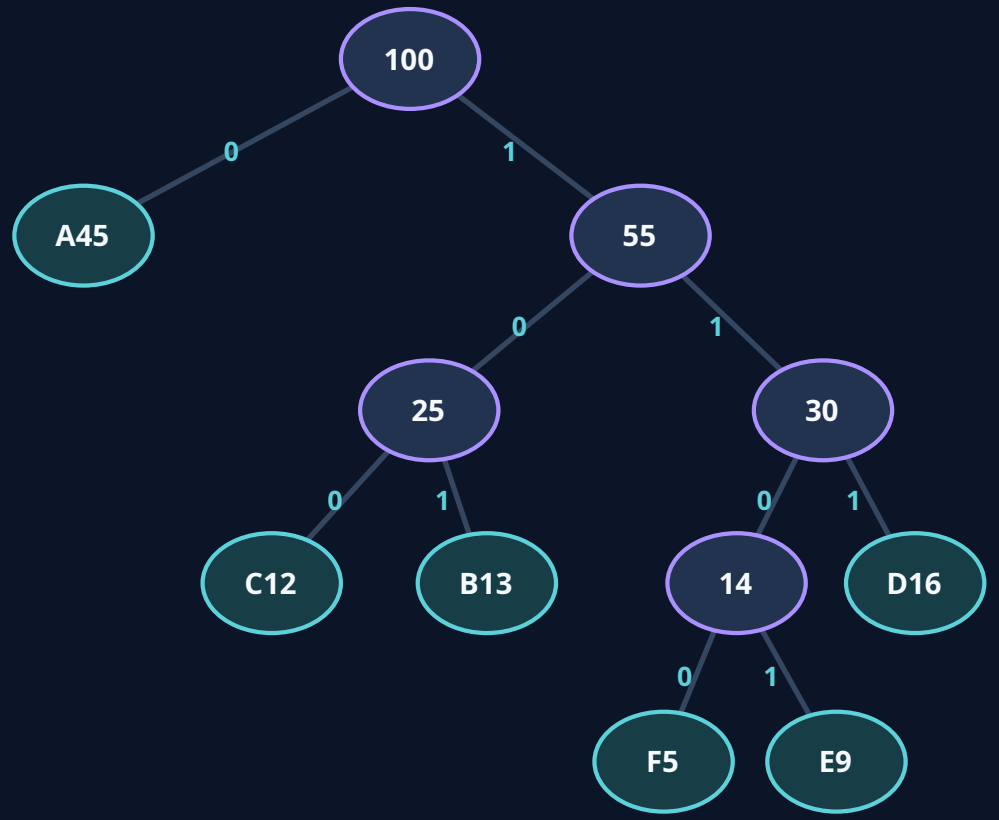
100 × H(X) = 221.988 theoretical bits

Huffman gap: only 2.012 bits above entropy bound.

Huffman: five min-heap merges

The two smallest subtrees are combined at each step.

01	F5 + E9	→ 14
02	C12 + B13	→ 25
03	14 + D16	→ 30
04	25 + 30	→ 55
05	**A45 + 55**	→ 100



Read each path root → leaf: left=0, right=1

Prefix-free codes make decoding unambiguous

Stage 3 codewords: A=0, B=101, C=100, D=111, E=1101, F=1100.

INPUT 0 1 0 1 0 1 1 0 0 1 1 1 0

0	101	0	1100	111	0
A	B	A	F	D	A

Decoded message: A B A F D A

No code is the prefix of another → every leaf ends exactly one symbol.

Canonical Huffman: deterministic codebook

With a known symbol alphabet/order, lengths determine canonical codes.

code_next = (code_previous + 1) << (new_length − previous_length)

SYMBOL	LENGTH	ASSIGNMENT	CANONICAL CODE
A	1	0	**0**
B	3	(0+1) << 2 = 4	**100**
C	3	(4+1) << 0 = 5	**101**
D	3	(5+1) << 0 = 6	**110**
E	4	(6+1) << 1 = 14	**1110**
F	4	(14+1) << 0 = 15	**1111**

224-bit payload unchanged. Standalone V4 stores symbol + length pairs; headers may be larger.

LZ77 sliding-window: seven emitted triples

S = ABRACADABRA_ABRACADABRA • W = 12 • L = 12 • “_” represents a space.

STEP	p	SEARCH BUFFER	TRIPLE (offset,len,next)	NEW p
1	0	—	(0,0,A)	1
2	1	A	(0,0,B)	2
3	2	AB	(0,0,R)	3
4	3	ABR	(3,1,C)	5
5	5	ABRAC	(2,1,D)	7
6	7	ABRACAD	(7,4,_)	12
7	**12**	**ABRACADABRA_**	**(12,10,A)**	**23**

Final step reserves one trailing “A” as next_char: 10 matched bytes + 1 literal.

Q5 — Time, memory, and compression capability

Encoder-focused bounds; N=input length, m=alphabet size, W=window, L=lookahead.

STAGE	ENCODE TIME	AUX SPACE	HEADER	BEST AT
V1 fixed	O(N + m)	O(m)	Alphabet / width	Simple baseline
V2 RLE	O(N)	O(1) streaming	Minimal	Long same-symbol runs
V3 tree	O(N + m log m)	O(m)	Tree structure	Skewed byte frequencies
V4 canon.	O(N + m log m)	O(m)	Symbol + length	Skewed bytes; varies
V5 LZ77	O(NWL + m′ log m′)	O(N+W+m′) batch	Several codebooks	Repeated multi-byte phrases

V5’s conservative worst-case LZ77 matching bound is O(NWL); faster indexed search in typical inputs.

Implementation: 20/20 exact round trips

Five encoders × four datasets; every decompressor reproduces its input byte-for-byte.

REQUIRED ROUND-TRIP TESTS

20 / 20 PASS

LONG RUNS

A×5000 + B×5000
10,000 bytes

SKEWED DNA

A 70%; C,G,T 10% each
12,000 bytes

PYTHON SOURCE

Pinned .py source snapshot
50 KiB

RANDOM BYTES

Pinned os.urandom sample
10,000 bytes

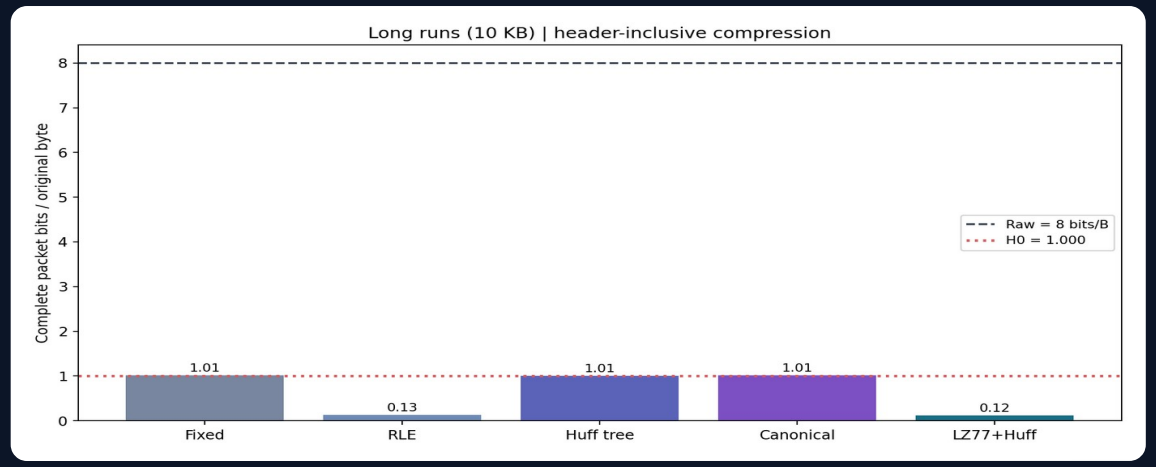
EXTRA TESTING

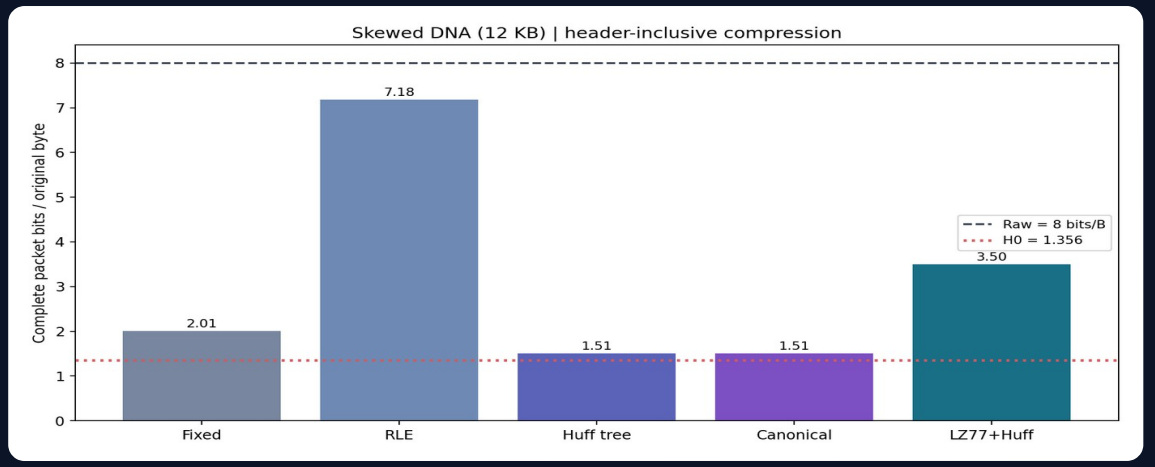
Empty input, one symbol, binary data, overlaps, 256 symbols.

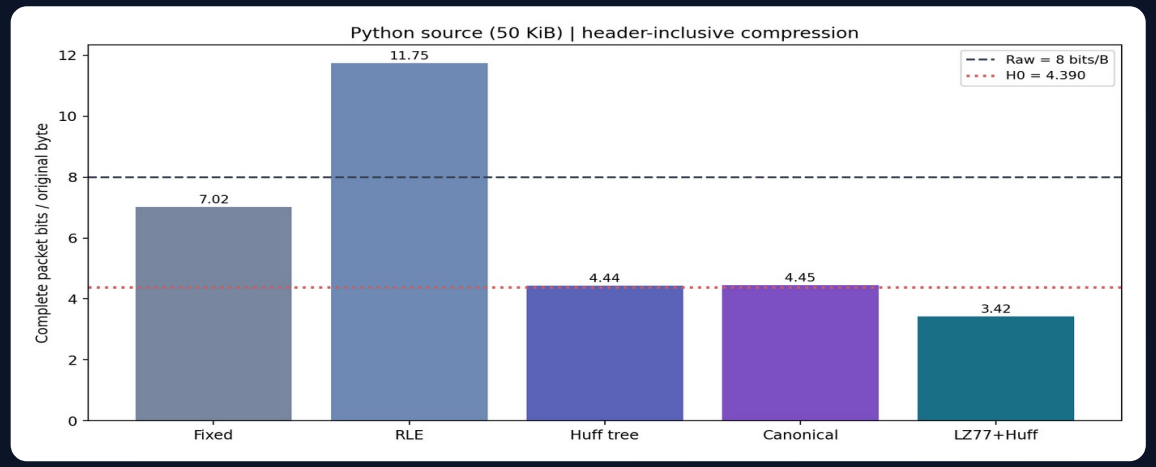
Metrics: packet bits/original byte (header included) + encoder/decoder throughput in MiB/s.

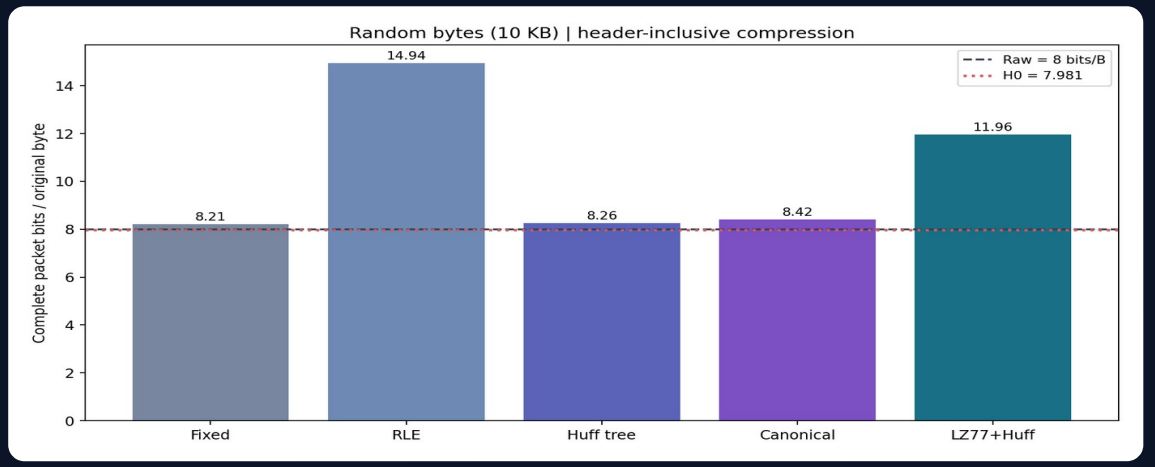
Compression ratio vs. entropy

Identical SHA256-pinned inputs; complete packet sizes include headers. Dashed=8.0; dotted=H_0.



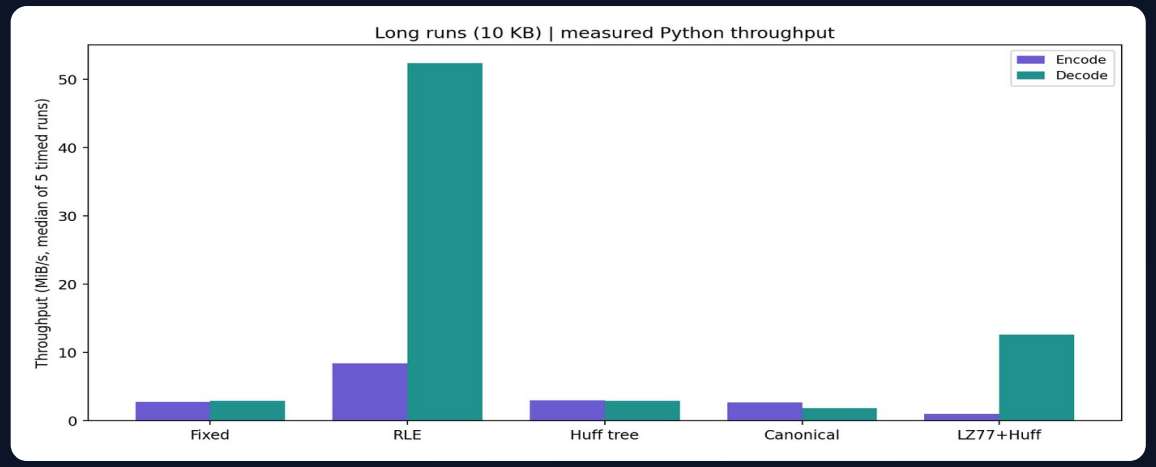


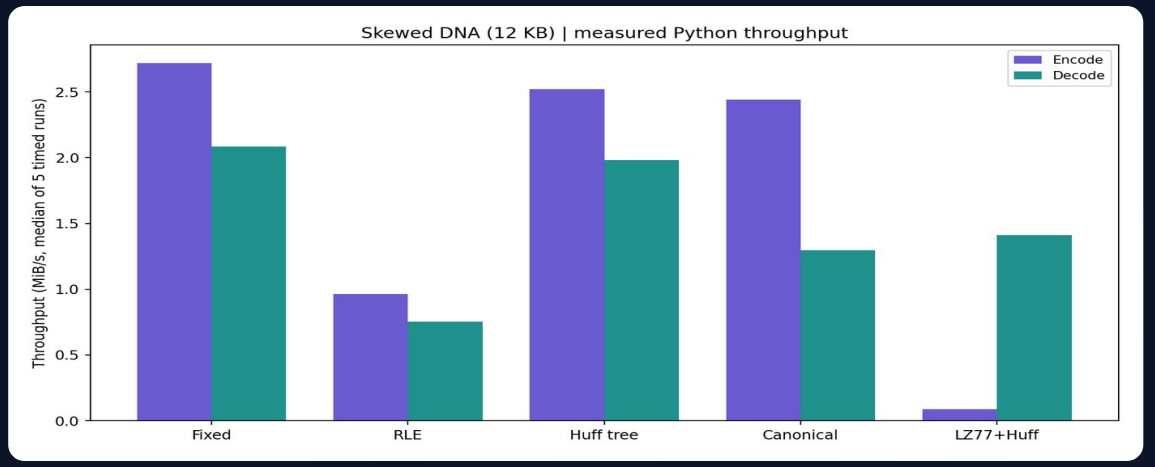


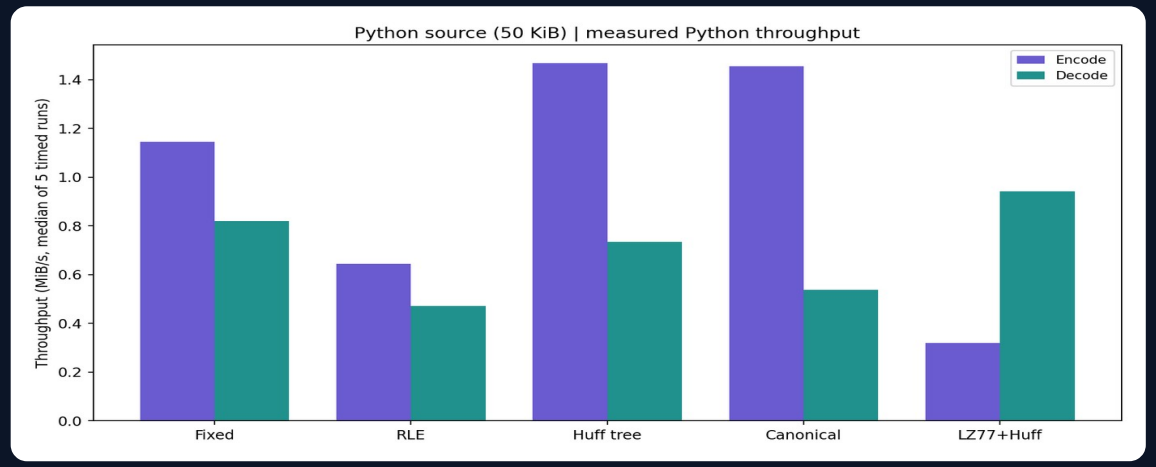


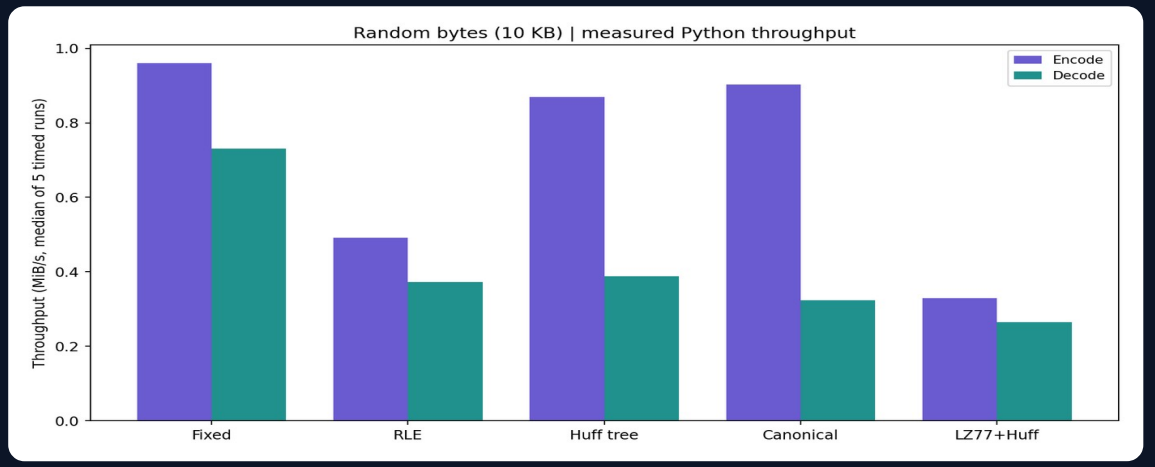
Encoding and decoding throughput

Median of 5 runs after 1 warm-up; input bytes pinned; rates depend on CPU and system load.









Conclusion & submission readiness

V4: verified corpus hashes, unified benchmarks, corrected Q4 space byte, and honest header tradeoffs.

EMPIRICAL RESULTS

Runs: RLE or LZ77
Skewed DNA: Huffman
Python source: LZ77+Huffman
Uniform random: generally expands

TECHNICAL LIMITATIONS

V4 metadata may exceed V3 tree metadata. Pinned random-input PACKETS: V4 10,520 B vs V3 10,326 B. V5 is custom, not RFC DEFLATE.

GITHUB: [REPLACE WITH YOUR VERIFIED REPOSITORY URL]

Still required: physically handwritten pages + group details + confirmed deadline + GitHub upload.